

K-means глазами новичка

K-means — один из самых понятных способов познакомиться с машинным обучением без учителя. Его задача простая по смыслу: есть набор точек без заранее заданных меток, и нужно разбить их на несколько групп так, чтобы внутри каждой группы точки были ближе друг к другу, а разные группы — заметно разделялись. В отличие от классификации, здесь нет правильных ответов, которые можно сравнить с прогнозом. Алгоритм сам ищет структуру данных по выбранному правилу похожести.

В учебном проекте каждая точка — это объект с двумя признаками, то есть вектор вида $x_i = (x_{i1}, x_{i2})$. Число групп задаётся параметром K . В приложении по умолчанию используется $K = 3$, но его можно менять в панели управления. Кнопка **Generate Points** создаёт новый набор данных, а **Start** запускает процесс кластеризации. Если открыть проект и ноутбук одновременно, становится заметно, как одинаковая идея выглядит и в интерактивном интерфейсе, и в более компактной библиотечной реализации.

Сам алгоритм строится вокруг центроидов — это точки, которые представляют собой «центр» каждого кластера. На первом шаге центроиды выбираются в данных. Затем каждая точка назначается ближайшему центроиду. После этого каждый центроид перемещается в среднее положение точек, которые к нему были назначены. Эти два действия повторяются до тех пор, пока центроиды перестают заметно менять своё положение. Именно поэтому K-means легко представить как цикл: назначить точки ближайшим центрам, пересчитать центры и повторять.

Если формально записать задачу, то для каждого кластера k есть множество точек S_k и центроид μ_k . Критерий качества выглядит так:

$$J = \sum_k \sum_{x_i \in S_k} \|x_i - \mu_k\|^2$$

Эта величина называется **SSE** (*sum of squared errors*). В коде проекта она используется как метрика, показывающая, насколько точки в среднем далеко от центров своих кластеров. Чем меньше SSE, тем плотнее и более согласованно устроены группы. Евклидово расстояние между точками считается по обычной формуле:

$$d(x_i, \mu_k) = \sqrt{(x_{i1} - \mu_{k1})^2 + (x_{i2} - \mu_{k2})^2}$$

Квадрат расстояния нужен потому, что он сильнее штрафует точки, которые находятся далеко от центра. Это делает метод чувствительным к разбросу внутри кластера и позволяет простой численной оптимизации двигать центры в нужную сторону.

Как работает цикл.

На каждом шаге алгоритм делает два повторяющихся действия. Сначала выполняется E-шаг: для каждой точки выбирается ближайший центроид. Формально это выглядит как поиск такого кластера k , для которого расстояние до центра минимально:

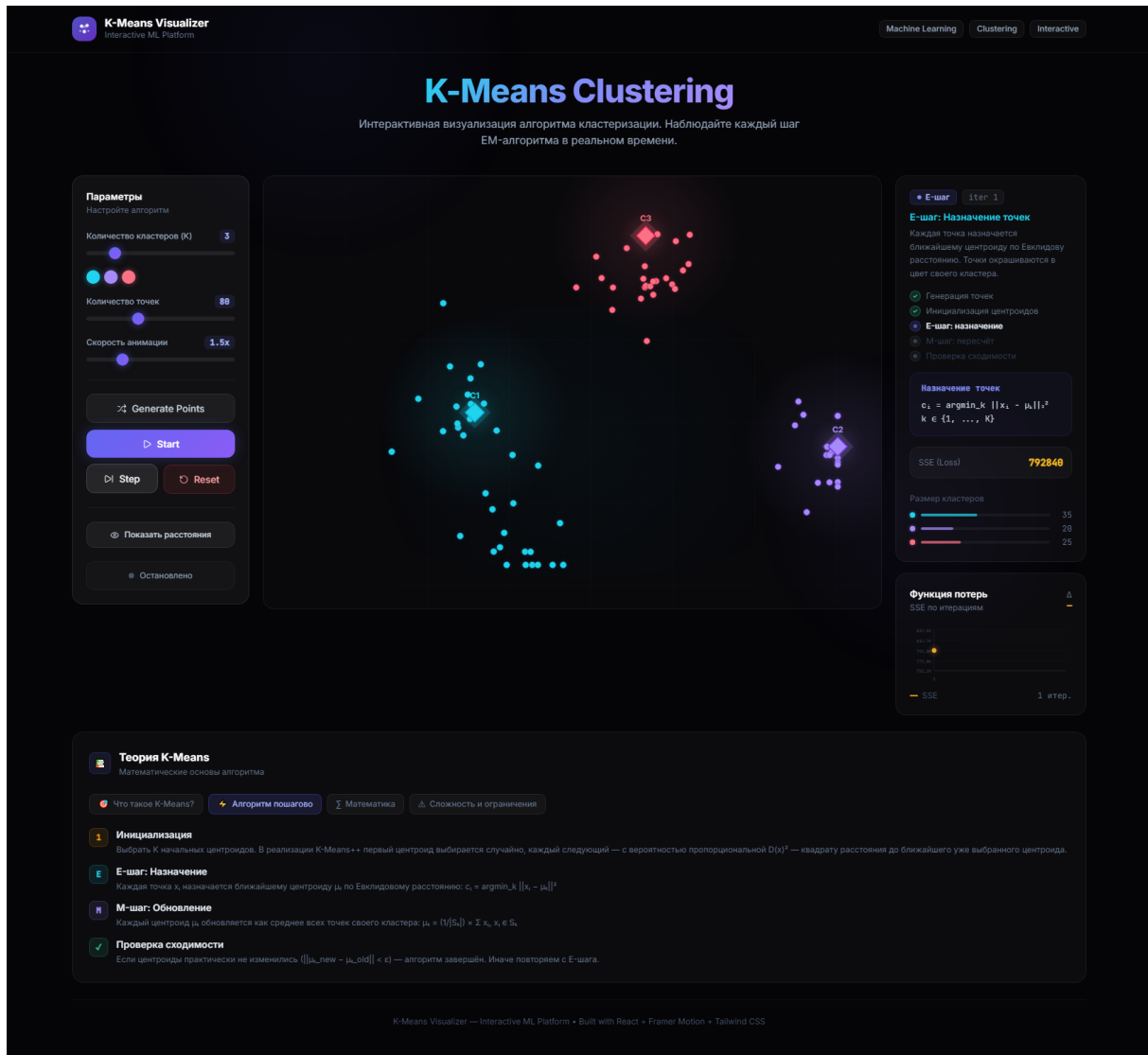


Рис. 1: На этой стадии приложение уже выполнило инициализацию и первый Е-шаг: каждая точка отнесена к ближайшему центроиду, а слева видны области, соответствующие разным кластерам. Справа отражены фаза процесса, размеры групп и текущее значение SSE.

$$c_i = \arg \min_k d(x_i, \mu_k)$$

В приложении этот этап соответствует фазе **Assign (Е-шаг)**: точки окрашиваются в цвет того кластера, к которому они были отнесены. Если включён показ расстояний, становится видно, как каждая точка «притягивается» к одному из центров. После этого следует М-шаг: для каждого непустого кластера новый центроид вычисляется как среднее по координатам всех точек в нём:

$$\mu_k^{new} = \frac{1}{|S_k|} \sum_{x_i \in S_k} x_i$$

В двумерном случае это просто два средних значения: отдельно по координате x и

отдельно по y . В проекте это реализовано в функции, которая обновляет центроиды и делает их перемещение заметным на графике. Стрелки на визуализации помогают увидеть, как центр кластера «переезжает» в более подходящее место после пересчёта.

Идея метода понятна интуитивно: в одном цикле точки выбирают ближайшие центры, а сами центры затем подстраиваются под эти точки. Важно понимать, что это не обучение на размеченных ответах, а процесс оптимизации выбранной функции качества. К тому же алгоритм не гарантирует глобально лучшего решения — он обычно находит локальный минимум. Это означает, что разные начальные положения центров могут дать разные, но вполне допустимые разбиения данных.

Почему важен старт.

Поэтому стартовый выбор центроидов имеет большое значение. В проекте используется инициализация по схеме **K-means++**. Сначала выбирается случайная первая точка, а затем каждый следующий центр выбирается с вероятностью, пропорциональной квадрату расстояния до ближайшего уже выбранного центра. Такая стратегия помогает центроиды не оказаться слишком близко друг к другу и лучше покрыть разные области данных. Это не устраняет случайность полностью, но обычно делает результат стабильнее, чем случайный запуск «с нуля».

Обычно в приложении смотреть на алгоритм удобнее всего пошагово. Сначала создаётся набор точек, затем задаётся небольшое число кластеров, например 3, после чего на каждом шаге можно наблюдать, как точки меняют назначение, а центроиды перемещаются. Такой режим особенно полезен для новичка, потому что позволяет задавать себе простые вопросы: «какой из центров сейчас ближе к этой точке?» и «почему после обновления центр должен сместиться именно сюда?». В этом смысле K-means — не просто формула, а визуальная история движения точек и центров.

Что происходит в ноутбуке.

Ноутбук **K-MeansJupyterNoteBook.ipynb** подходит для того, чтобы увидеть тот же процесс в более компактной и библиотечной форме. Там сначала создаются синтетические данные через `make_blobs`, затем вызывается `sklearn.cluster.KMeans(n_clusters=3, random_state=42)`, а в финальной ячейке рисуются кластеры и центроиды. Визуально это выглядит очень компактно, но именно React-приложение помогает понять логику происходящего: как выбираются ближайшие центры, как они смещаются после пересчёта, как останавливается процесс и по каким сигналам видно, что алгоритм нашёл устойчивое решение.

С точки зрения вычислительной сложности, при числе точек n , числе кластеров K , размерности признаков d и числе итераций t типичная оценка работы K-means составляет $O(n \times K \times d \times t)$. Для двумерного случая это очень быстро, но на больших наборах данных цена становится заметной. Метод чувствителен к нескольким важным ограничениям: число кластеров нужно задать заранее, выбросы могут сильно сдвигать центры, а компактные кластеры примерно сферической формы обычно подходят лучше, чем сильно вытянутые или пересекающиеся группы. Кроме того, масштаб признаков влияет на результат: признак с большим диапазоном значений будет сильнее влиять на расстояние, чем более мелкий.

Поэтому на практике K-means часто запускают несколько раз с разными начальными центроидами, масштабируют признаки и выбирают K так, чтобы полученное разби-

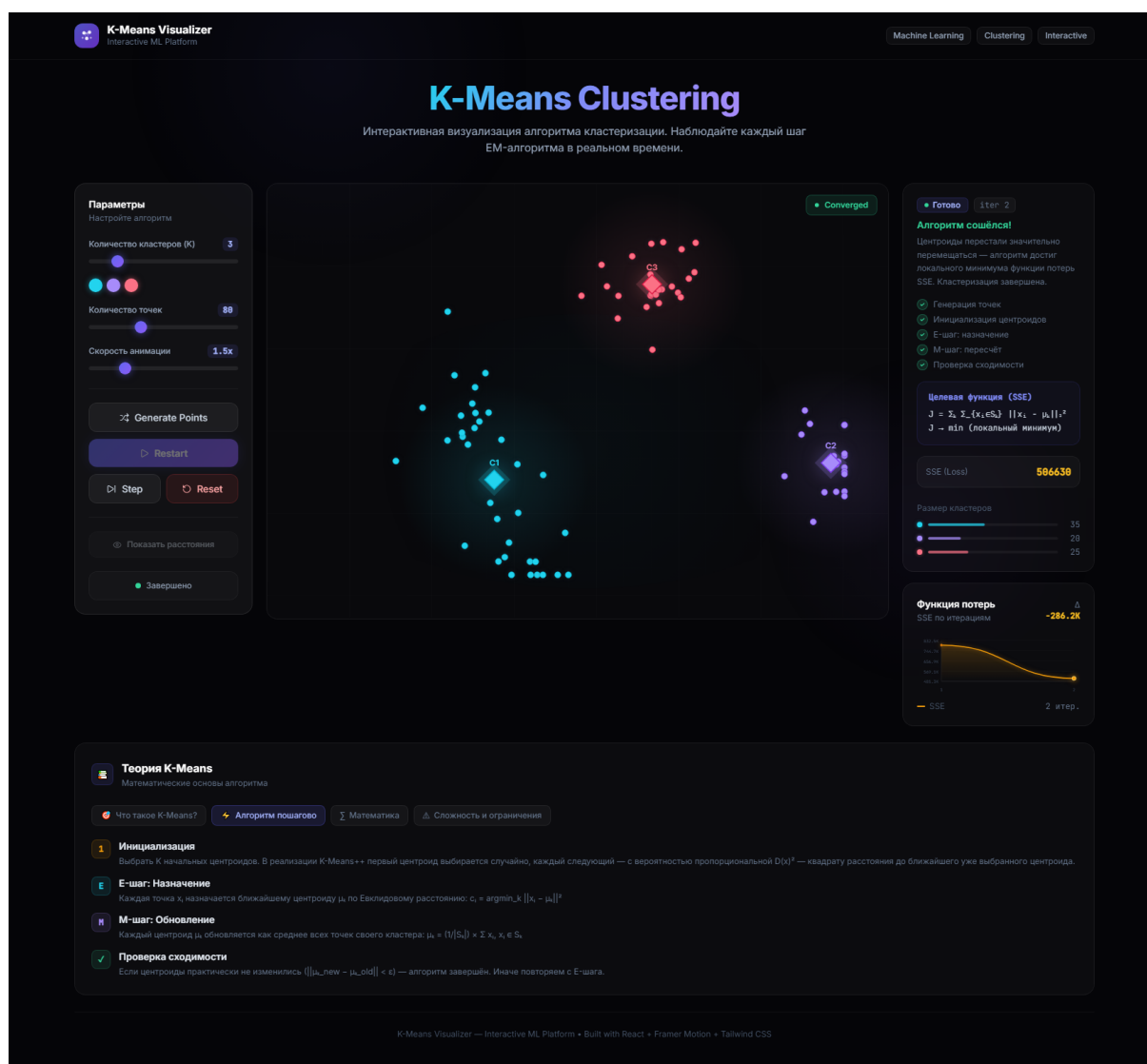


Рис. 2: После продолжения вычислений центроиды перемещаются к средним точкам своих групп, и график SSE постепенно уменьшается. В момент стабилизации система показывает, что алгоритм сошёлся.

ение было не просто математически устойчивым, а содержательно осмысленным. В проекте число кластеров можно менять вручную, а график SSE помогает сравнивать разные варианты и понимать, насколько заметно изменился результат. Часто в таких задачах используется метод локтя: строят зависимость SSE от числа кластеров и находят точку, после которой добавление новых групп уже не даёт заметного улучшения. Это не универсальная формула, но хороший практический ориентир.

Важно отметить, что K-means — это не «магический» способ быстро находить правильные группы в любых данных. Его сильная сторона в простоте и наглядности: за короткий цикл из двух шагов он умеет находить структуру в данных даже без меток. Слабая сторона — в том, что итог зависит от выбранного K , от начальных центров и от формы самих кластеров. Именно поэтому этот алгоритм лучше всего понимать не как готовую модель «на все случаи», а как простой и удобный инструмент для исследования данных, который виден на экране и легко можно проверить на примере.

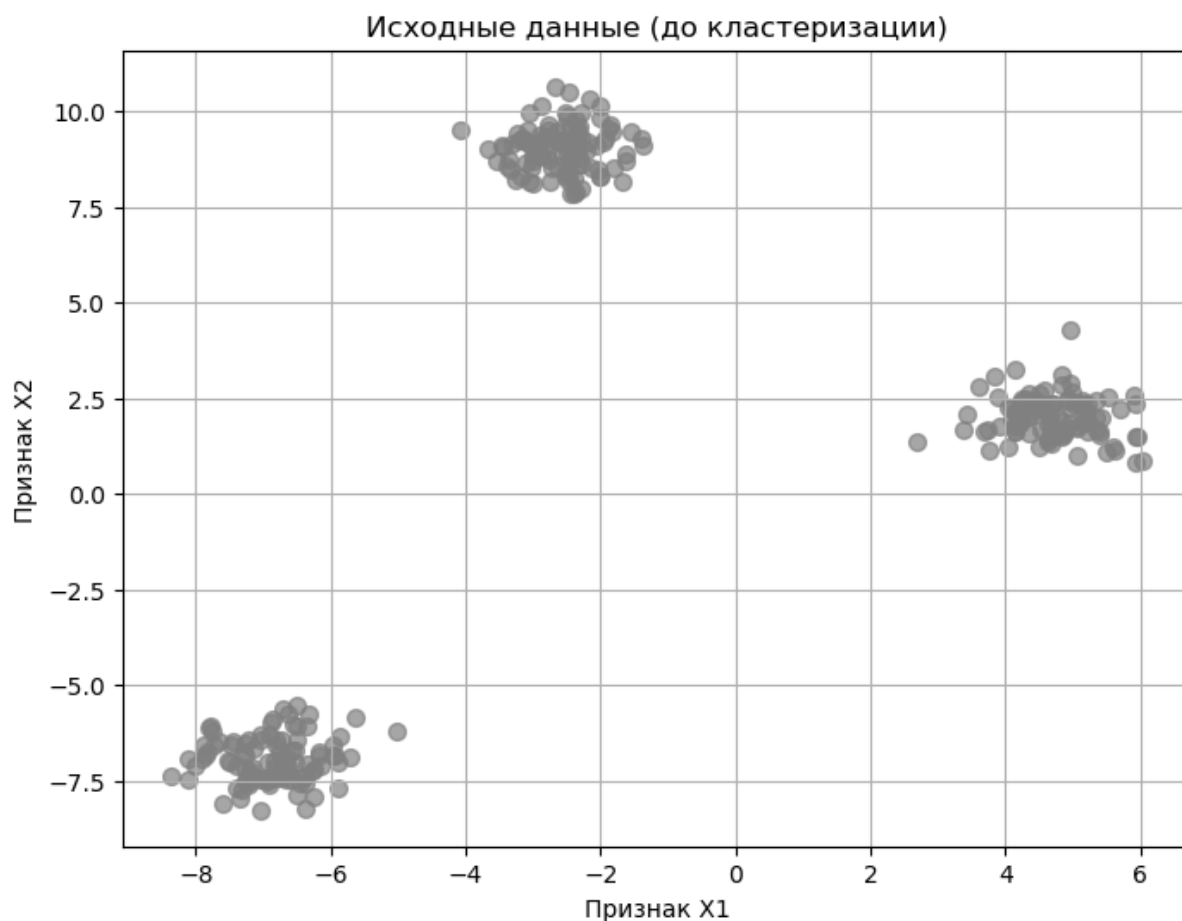


Рис. 3: В ноутбуке данные создаются искусственно: точки формируются в несколько компактных облаков, чтобы задача кластеризации была наглядной.

Полезные материалы.

- Skillfactory. *Алгоритм k-means и метод локтя: кластеризация данных с примерами на Python*. Habr: <https://habr.com/ru/companies/skillfactory/articles/877684/>
- Википедия. *Метод k-средних*. https://ru.wikipedia.org/wiki/РѢРхСЪР«РҮ_k-СҒСГРхРҮР,,РчС
- YouTube. *Объяснение алгоритма K-means*: <https://www.youtube.com/watch?v=uw9XHsDKcXs>

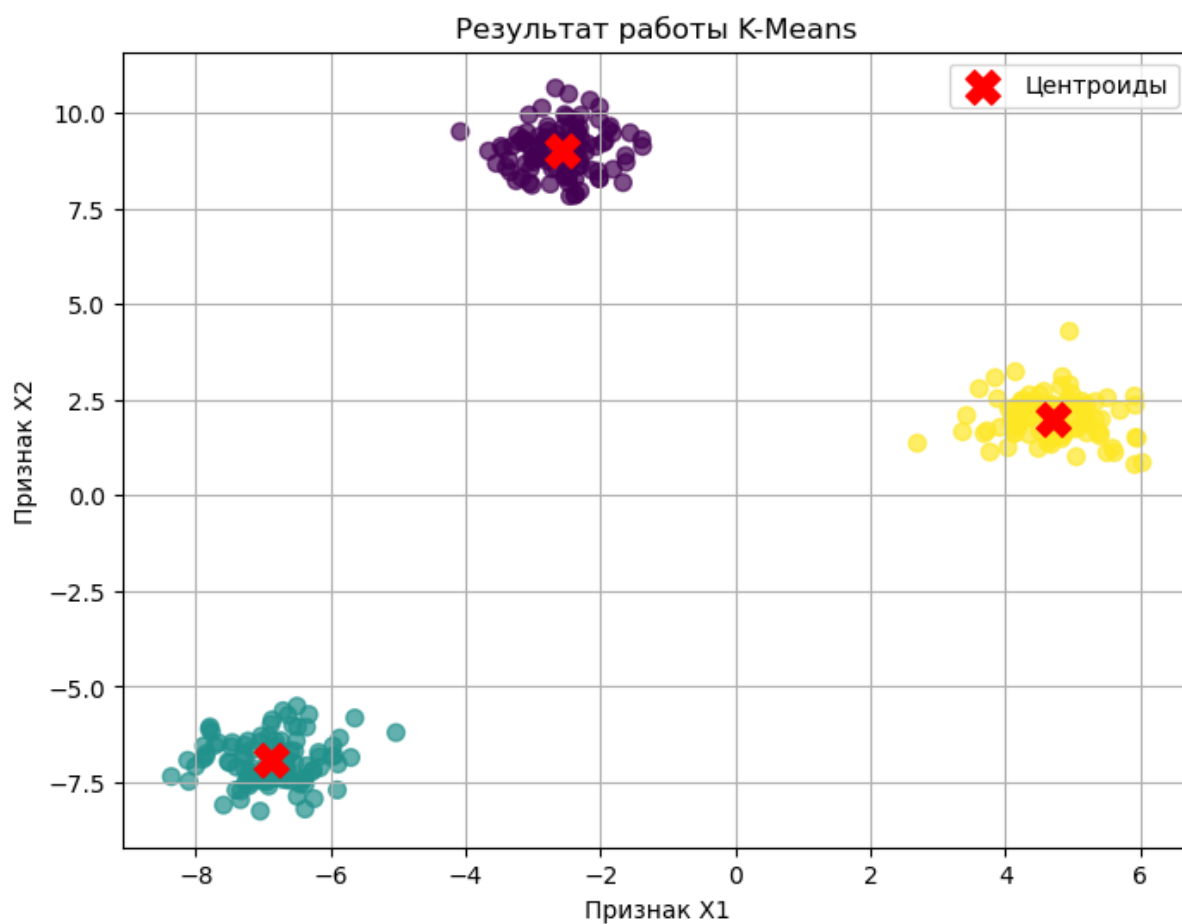


Рис. 4: В результате работы `sklearn.KMeans` точки окрашиваются по кластерам, а красные крестики обозначают найденные центроиды.